Implementação de um Motor de Pré-processamento e Refino de Dados em Python

Andrey Mota de Oliveira[1], Claudio dos Santos Junior[1], Gabriel da Silva e Silva[1], Louise de Souza Santino[1]

Sistemas da Informação – Centro Universitário de Excelência (UNEX)
Feira de Santana – BA – Brasil

{deymotta686,claudiosjrel, gabrielsilva82374055}@gmail.com, louisesantino@hotmail.com

Resumo. *Este trabalho expande uma biblioteca estatística nativa em linguagem de programação Python com um módulo de pré-processamento de dados. Foram implementadas as classes MissingValueProcessor, Scaler e Encoder para o tratamento de valores ausentes, escalonamento e codificação de variáveis. O projeto introduziu o isolamento de colunas para garantir a compatibilidade com a classe Statistics, evitando erros por tipos mistos durante o processamento. A robustez do sistema foi validada por testes unitários e aplicada em um dataset real do Spotify. Operações como dropna e fillna, MinMax e Z-Score, prepararam os dados para tarefas de mineração. Os resultados confirmam a viabilidade técnica da solução.*

Abstract. *This work expands a native statistical library in a programming language with a data preprocessing module. The classes MissingValueProcessor, Scaler, and Encoder were implemented for handling missing values, feature scaling, and variable encoding. The project introduced column isolation to ensure compatibility with the Statistics class, preventing mixed-type errors during processing. The system's robustness was validated through unit tests and applied to a real-world Spotify dataset. Operations such as dropna and fillna, along with Min-Max and Z-Score scaling, prepared the data for mining tasks. The results confirm the technical feasibility of the solution.*

1. Introdução

O pré-processamento é a etapa mais crítica da Mineração de Dados, pois transforma dados brutos em ativos de informação consistentes. Sua função é eliminar ruídos e inconsistências que elevam o custo computacional e degradam a performance dos algoritmos. O projeto da **Dende Softhouse** automatiza essa preparação, oferecendo ferramentas para o tratamento de valores ausentes e a mitigação de *outliers*, garantindo que o cliente consuma apenas dados de alta qualidade.

A proposta consistiu em implementar, manualmente, algoritmos de limpeza, escalonamento e codificação em linguagem de programação Python, sem o uso de bibliotecas auxiliares como *Pandas* ou *Scikit-Learn*. Essa abordagem assegura o controle total sobre a manipulação da estrutura de dados e consolida a compreensão lógica necessária para transformar dados brutos em entradas válidas para modelos preditivos. Ao abdicar de abstrações prontas, o projeto garante a precisão matemática das transformações e otimiza a alocação de recursos nas análises.

2. Fundamentação Teórica

A base conceitual deste projeto fundamenta-se no ciclo de vida KDD (*Knowledge Discovery in Databases*). Segundo Pedregosa et al. (2011), "a análise preditiva de dados exige ferramentas robustas e eficientes para o pré-processamento e aprendizado de máquina, integradas em uma linguagem de alto nível como Python para garantir acessibilidade e desempenho". Entende-se, assim, que o pré-processamento precede a mineração e é vital para o sucesso do modelo final.

2.1. Integridade e Qualidade dos Dados (GIGO)

O projeto baseia-se no princípio *Garbage In, Garbage Out*. A teoria justifica o uso de métodos de limpeza para mitigar ruídos e enviesamentos, garantindo que a amostra final seja estatisticamente representativa.

2.2. Tratamento de Dados Ausentes (*Missing Values*)

Dentro desse tratamento basicamente entendemos que dados nulos podem interromper o fluxo de algoritmos ou distorcer métricas. Enquanto os "isna" e "notna" verificam e validam os dados:

- Is Not a Number (*isna*): Verifica a existência de lacunas. Se o dado for *NaN*, vazio ou *None* ele retorna *True* em relação as ausências.
- Is Not Not a Number (*notna*): Verificação oposta. Se o campo estiver preenchido, ele retorna *True* para o preenchimento.

Os seguintes métodos funcionam como "agentes transformadores" que lidam com as informações, se baseando nas primeiras leituras.

- Exclusão (*dropna*): Remoção de registros incompletos.
- Imputação (*fillna*): Substituição de lacunas por constantes ou medidas de tendência central (Média/Moda) para preservar o tamanho da amostra.

2.3. Escalonamento de Atributos ajusta Variáveis de diferentes magnitudes para uma escala comum.

- Normalização Min-Max: Mapeia dados para o intervalo [0,1]. Essa modificação de escala possibilita um melhor tratamento de dados grandes e *outlier*, que são dados distanciados e discrepantes.

$$x_{norm} = max(X) - min(X) / x - min(X)$$

Onde x_{norm} é o valor normalizado, x representa o dado original, min(X) e max(X) são os valores mínimo e máximo da coluna, respectivamente.

- Padronização Z-Score: Centraliza os dados na média (μ) com unidade de desvio padrão (σ).

$$z = \sigma x / \mu$$

z: o escore-z resultante;
σ: o desvio padrão da coluna.

x: o valor original do atributo;
μ: a média aritmética da amostra;

2.4. Representação em Espaço Vetorial

O *encode()* fundamenta a conversão de dados nominais em numéricos.

One-Hot Encoding: Transforma categorias em dimensões binárias (0 e 1) através da criação de categorias únicas para cada categoria específica. Onde o dado entra na categoria ele determina 1, as demais, ficam em 0. Isso evita que o sistema assuma ordens de importância inexistentes entre palavras (ex: gêneros musicais).

3. Metodologia

O módulo de pré-processamento foi desenvolvido utilizando o paradigma de Orientação a Objetos em linguagem de programação – sem dependências externas. E a arquitetura foi dividida em classes especialistas (MissingValueProcessor, Scaler e Encoder), coordenadas por uma classe de fachada (Preprocessing), garantindo modularidade e facilidade de manutenção. Segundo Wilson et al. (2014), "as boas práticas de computação científica priorizam a escrita de programas voltados para pessoas, e não apenas para máquinas, com foco em um código modular, legível e testado para garantir a confiabilidade da pesquisa".

3.1. Validação Estrutural e Integridade

No método construtor *__init__* da classe *Preprocessing*, implementamos a função *_validate_dataset_shape*. Esta metodologia assegura que o dataset (tipo *dict*) contenha colunas (tipo *list*) com comprimentos idênticos. O objetivo é evitar erros de índice durante o processamento de matrizes, garantindo a consistência necessária para as etapas subsequentes do ciclo KDD.

3.2. Tratamento de Valores Ausentes

A classe *MissingValueProcessor* utiliza lógica de varredura por linha para identificar a presença de *None*.

- Remoção (*dropna*): É o método que identifica índices que contêm valores nulos em colunas alvo e reconstrói o dicionário apenas com as linhas íntegras.
- Imputação (*fillna*): É o método que aplica uma verificação de tipo em tempo real. Se a coluna for numérica, o valor de preenchimento é convertido para *float* antes da inserção, mantendo a homogeneidade dos dados para cálculos estatísticos.

3.3. Escalonamento e Isolamento Atômico

A metodologia de escalonamento utiliza a classe *Statistics* de forma isolada por coluna, técnica denominada instanciação atômica.

- Min-Max: Calcula-se a amplitude dos dados (max−min) para mapear os valores no intervalo [0,1].

- Z-Score: Para cada coluna numérica, o *Scaler* instância um objeto *Statistics* temporário. Isso permite extrair a média (μ) e o desvio padrão (σ) de forma segura, isolando colunas de texto que poderiam causar exceções de tipo durante o cálculo da

biblioteca original.

3.4. Transformação Categórica e Esparsidade

A classe *Encoder* gerencia a conversão de dados nominais para o espaço numérico.

- Label Encoding: Atribui um identificador inteiro único para cada categoria através de um mapeamento de dicionário ordenado.
- One-Hot Encoding: Implementa a expansão de dimensionalidade. O método utiliza *.pop()* para extrair a coluna original e insere novas chaves binárias no dataset, onde cada chave representa uma categoria (ex: *genre_Pop*), populada com valores 0 ou 1.

3.5. Orquestração e Sincronização de Estado

A classe *Preprocessing* atua como orquestradora. O método *_update_stats_safely* é executado após qualquer alteração estrutural no dataset (como remoção de nulos). Ele tenta reinstanciar o objeto de estatísticas global dentro de um bloco *try-except*, garantindo que as métricas de média e desvio padrão estejam sempre sincronizadas com o estado atual dos dados "limpos", sem interromper o fluxo de execução em caso de erros de tipagem.

3.6. Ajustes dos Testes Unitários

Diferente da etapa anterior do projeto, nesta implementação do módulo de pré-processamento não se fez necessária nenhuma alteração nos parâmetros ou nos dados de entrada do arquivo *test_preprocessing.py*. Este fato indica uma evolução na robustez do sistema, uma vez que o código atual foi projetado para tratar preventivamente as não conformidades estruturais e estatísticas.

3.7. Aplicação do preprocessamento no dataset Spotify

A validação prática da biblioteca foi realizada via script *analise_preprocessing_spotify.py*, processando o arquivo *track_data_final.csv* em quatro etapas fundamentais. Inicialmente, executou-se o carregamento com tipagem dinâmica, convertendo numerics e mapeando lacunas como None. Na sequência, a limpeza por imputação utilizou o método fillna (valor constante 0) para garantir a integridade estatística. Para homogeneizar grandezas distintas, aplicou-se a normalização Min-Max, reduzindo os atributos ao intervalo [0,1]. Por fim, a codificação categórica (*Label* e *One-Hot Encoding*) converteu metadados textuais em vetores numéricos, transformando o dataset bruto em uma estrutura estritamente matemática apta para mineração.

4. Resultados e Discussões

A validação do módulo de pré-processamento foi dividida em testes de unidade para garantia lógica e aplicação prática em um dataset real de larga escala.

4.1. Resultado de Teste Unitário

No teste de integração da classe *Preprocessing*, validou-se o fluxo completo de transformação. Em uma amostra de teste com valores nulos e categorias, o método *dropna* removeu corretamente os registros incompletos, enquanto o *minMax_scaler* converteu uma

lista de entradas arbitrárias para o intervalo exato [0,1]. Os resultados coincidiram com os valores esperados, validados via unittest com *assertAlmostEqual* para as métricas de ponto flutuante, confirmando a estabilidade da biblioteca.

4.2. Tratamento de Nulidade no Dataset Spotify

A análise foi aplicada ao arquivo *track_data_final.csv*, contendo 8.582 músicas. O diagnóstico inicial via *isna()* revelou lacunas em colunas críticas:

- *track_popularity: 12 nulos detectados.*
- *artist_followers: 45 nulos detectados.*
- *album_total_tracks: 2 nulos detectados.*

Após a execução do método *fillna(value=0)*, o sistema confirmou a imputação total, convertendo os tipos dinamicamente para *float*. Essa etapa foi fundamental para permitir que as operações matemáticas subsequentes não fossem interrompidas por exceções de tipo.

4.3. Escalonamento e Normalização

Aplicou-se a métrica Min-Max para normalizar as colunas de popularidade e duração, que possuíam ordens de magnitude distintas.

1. Popularidade (0-100): Antes: [0.00,100.00] | Depois: [0.00,1.00].
2. Duração (ms): Antes: [30.000,1.200.000] | Depois: [0.00,1.00].

- Interpretação: A técnica eliminou o viés de magnitude, permitindo que atributos com valores numéricos altos (milissegundos) não dominassem algoritmos de mineração sobre atributos de escala menor (popularidade).

4.4. Codificação Categórica e Esparsidade

As variáveis qualitativas foram submetidas ao Encoder para transformação em espaço vetorial:

- Label Encoding (explicit): Os valores booleanos foram convertidos para [0,1], permitindo correlações numéricas diretas.
- One-Hot Encoding (album_type): A coluna original foi expandida em três novas dimensões binárias: *album_type_album*, *album_type_single* e *album_type_compilation*.
- Discussão: A técnica de expansão garantiu que categorias nominais fossem representadas sem hierarquia matemática inexistente, preservando a integridade posicional de cada faixa no dataset.

4.5. Comparação Conceitual com Bibliotecas

A implementação da Dende Softhouse utiliza o método *.pop()* para garantir a eficiência de memória durante o One-Hot Encoding, removendo a coluna original assim que as novas são criadas. Diferente de bibliotecas como o *Pandas* (*get_dummies*), que criam cópias completas do DataFrame e podem dobrar o consumo de RAM, nossa abordagem foca no processamento *in-place* dentro do dicionário nativo, otimizando recursos em

ambientes de hardware limitado.

5. Considerações Finais

5.1. Pontos Positivos e Negativos dos Métodos Utilizados

MissingValueProcessor (Tratamento de Nulos)

Este método, por meio do *fillna*, evita o travamento em cálculos matemáticos ao preencher e "homogeneizar" os dados, o que permite preservar o volume total do dataset. No entanto, o uso do *dropna* pode ocasionar perda significativa de informações caso existam muitos registros nulos. Além disso, o próprio *fillna* pode introduzir um leve viés estatístico ao inserir valores predefinidos onde originalmente não havia dados.

Scaler (Min-Max e Z-Score)

Este módulo oferece duas abordagens de normalização. O Min-Max projeta os dados em um intervalo fixo (geralmente entre 0 e 1), o que é ideal para algoritmos que não processam valores negativos; contudo, ele apresenta alta sensibilidade a *outliers*, que podem comprimir os demais valores em uma faixa excessivamente estreita próxima a zero. Já o Z-Score demonstra maior robustez contra valores extremos ao utilizar o desvio padrão como métrica de escala, mantendo a integridade da distribuição estatística. O ponto de atenção desta técnica é a geração de números negativos, que podem ser incompatíveis com modelos específicos de mineração de dados.

Encoder (Label e One-Hot)

O Label Encoding destaca-se pela simplicidade e por não expandir a dimensionalidade do *dataset*, mantendo a estrutura original das colunas. Seu principal risco, contudo, é a introdução de uma "falsa hierarquia", onde o algoritmo pode interpretar erroneamente uma categoria numérica maior como superior a outra. Em contrapartida, o One-Hot Encoding é a solução ideal para variáveis nominais sem ordem intrínseca (como gêneros musicais), eliminando vieses hierárquicos. Seu ponto negativo reside no elevado consumo de memória e processamento, devido à criação de múltiplas novas colunas para cada categoria mapeada.

Preprocessing (Classe Orquestradora)

A principal vantagem desta classe é a centralização e automação de todo o fluxo de preparação de dados. Ela assegura a integridade das análises ao atualizar as estatísticas internas automaticamente após cada etapa de limpeza (via *_update_stats_safely*). Por outro lado, essa arquitetura centralizada cria uma alta dependência: qualquer falha lógica na classe orquestradora pode interromper, em cascata, todos os processos subsequentes de limpeza, codificação e escalonamento.

5.2. Conclusão Final

O objetivo do projeto de implementação do preprocessamento foi atingido: oferecer aos clientes da **Dende Softhouse** as melhores e mais importantes informações, eliminando ruídos. Através da implementação da classe em Python Nativo, enxergamos a real viabilidade do projeto. Levando em consideração que a parte referente aos dados

estatísticos foi reaproveitada, foi relativamente simples dar continuidade ao projeto.

Os nossos principais desafios residiram na manutenção da integridade referencial do dataset durante operações de remoção e na gestão manual da tipagem, uma vez que a ausência de bibliotecas de alto nível exigiu a implementação de validadores de forma para assegurar que as transformações matemáticas fossem aplicadas sobre estruturas de dados consistentes e sincronizadas.

Os testes práticos com o dataset do Spotify demonstraram que a biblioteca é capaz de lidar com as imperfeições inerentes aos dados reais, como valores ausentes e escalas heterogêneas, sem depender de pacotes externos. Conclui-se que a estrutura implementada cumpre os requisitos de rigor matemático e eficiência computacional necessários para a etapa de preparação de dados, fornecendo uma base sólida e limpa para as fases futuras de modelagem preditiva e descoberta de conhecimento.

4. Referencias

KAGGLE. Spotify Songs for ML & Analysis (8700+ tracks). **Dataset de AlyAhmedTS13**. Disponível em: https://www.kaggle.com/datasets/alyahmedts13/spotify-songs-for-ml-and-analysis over-8700-tracks. Acesso em: 15 mar. 2026

PEDREGOSA, F., VAROQUAUX, G. AND GRAMFORT, A. (2011) "Scikit-learn: Machine Learning in Python", **Journal of Machine Learning Research**, Disponível em: https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html, Acesso em: 15 mar. 2026.

WILSON, G. ET AL. (2014) "Best Practices for Scientific Computing", **PLOS Biology**, Disponível em: https://journals.plos.org/plosbiology/article?id=10.1371/journal.pbio.1001745, Acesso em: 18 mar. 2026.